

Software Edificio Miraflores

Documento Arquitectura de Software (DAS) Versión 1.0

Integrantes: Guillermo León Schroeder Toledo

Profesor: Ricardo Alfonso Pino Aranda

Asignatura: Ingeniería de Software

Sección: 003D

Identificación de Documento

Identificación	001
Proyecto	Software de automatización de administración y control de gastos comunes
Versión	1.0.2

Documento mantenido por	Guillermo Leon Toledo Schroeder
Fecha de última revisión	2026-06-14
Fecha de próxima revisión	—

Documento aprobado por	Ricardo Alfonso Pino Aranda
Fecha de última aprobación	—

Historia de Revisiones

Fecha	Versión	Descripción	Autor
03 - 04 - 2026	1.0.0	el punto 1.1 hasta 2.5	Guillermo Schroeder
25-5-2026	1.0.1	punto 3 hasta 4.5.3	Guillermo Schroeder
14-6-2026	1.0.2	punto 5 hasta punto 9	Guillermo Schroeder

Tabla de Contenidos

Identificación de Documento	2
1. INTRODUCCIÓN	5
1.1. CONTEXTO DEL PROBLEMA	5
1.2. PROPÓSITO	6
1.3. ÁMBITO	6
1.4. DEFINICIONES, ACRÓNIMOS Y ABREVIACIONES	7
1.5. RESUMEN EJECUTIVO	7
1.6. ARQUITECTURA DEL SISTEMA	8
2. VISIÓN DEL SISTEMA	8
2.1. DESCRIPCIÓN GENERAL DEL SISTEMA	8
2.2. OBJETIVOS DEL SISTEMA	8
2.3. REQUERIMIENTOS FUNCIONALES	9
2.4. REQUERIMIENTOS NO FUNCIONALES	10
2.5. SUPUESTOS Y DEPENDENCIAS	11
3. ESTILOS Y PATRONES ARQUITECTÓNICOS	11
3.1. Estilo arquitectónico adoptado	11
3.2. JUSTIFICACIÓN DEL ESTILO SEGÚN EL CONTEXTO DEL SISTEMA	13
3.3. Patrones de diseño aplicados	13
4. MODELO 4 +1 Y VISTAS ARQUITECTÓNICAS	15
4.1. VISTA DE ESCENARIO	15
4.1.1. <i>Propósito</i>	15
4.1.2. <i>Actores</i>	15
4.1.3. <i>Diagrama general de casos de uso</i>	17
4.1.4. <i>Diagrama de casos de uso específicos</i>	17
4.1.5. Lista de casos de uso	17
4.1.6. <i>Especificación de casos de uso</i>	18
4.2. VISTA LÓGICA	19
4.2.1. <i>Propósito</i>	19
4.2.2. <i>Diagrama de clases</i>	19
4.2.3. <i>Descripción diagrama de clases</i>	19
4.3. VISTA DE IMPLEMENTACIÓN/DESARROLLO	21
4.3.1. <i>Propósito</i>	21
4.3.2. <i>Diagrama de componente</i>	21
4.3.3. <i>Descripción diagrama de componente</i>	21
4.3.4. <i>Diagrama de paquete</i>	21
4.3.5. <i>Descripción diagrama de paquete</i>	21
4.4. VISTA DE PROCESOS	22

4.4.1. PROPÓSITO	22
4.4.2. DIAGRAMA DE ACTIVIDAD	22
4.4.3. DESCRIPCIÓN DIAGRAMA DE ACTIVIDAD	22
4.5. VISTA FÍSICA	23
4.5.1. <i>Propósito</i>	23
4.5.2. <i>Diagrama de despliegue</i>	23
4.5.3. <i>Descripción diagrama de despliegue</i>	23
5. REQUISITOS DE CALIDAD	24
5.1. PROPÓSITO	24
5.2. Atributos de calidad	24
5.3. <i>Reglas y criterios de evaluación de calidad</i>	26
6. PRINCIPIOS DE DISEÑO APLICADOS	28
6.1. <i>Propósito</i>	28
6.2. PRINCIPIOS DE DISEÑO	28
7. PROTOTIPO	29
7.1. PROPÓSITO	29
7.2. MOCKUPS (IMÁGENES CON UNA BREVE DESCRIPCIÓN)	29
7.3. JUSTIFICAR HERRAMIENTAS DE PROTOTIPADO	31
8. EVALUACIÓN DE CALIDAD HEURÍSTICA DE NIELSEN	31
8.1. PROPÓSITO	31
8.2. LISTA DE VERIFICACIÓN	32
8.3. ANÁLISIS Y MÉTRICAS DE RESULTADOS	33
9. CONTROL DE VERSIONES	33
9.1. PROPÓSITO	33
9.2. CONTROL DE VERSIÓN UTILIZADO	33
9.3. JUSTIFICAR HERRAMIENTAS DE VERSIONAMIENTO	34
10. CONCLUSIONES	35
11. BIBLIOGRAFÍA	35
12. ANEXOS	36
12.1. CARTA GANTT	36

1. INTRODUCCIÓN

1.1. Contexto del Problema

El Edificio Residencial Miraflores enfrenta actualmente dificultades en la administración y control de los gastos comunes debido a que gran parte de los procesos asociados a pagos, consultas y seguimiento de información financiera se realizan de forma manual. Los residentes deben acudir presencialmente a la administración para consultar estados de

cuenta, verificar pagos realizados o solicitar información relacionada con sus obligaciones económicas.

Esta situación genera retrasos en la entrega de información, aumenta la carga operativa del administrador y dificulta la transparencia en la gestión financiera de la comunidad. Asimismo, la inexistencia de herramientas digitales limita la capacidad de generar reportes consolidados y dificulta la identificación temprana de situaciones de morosidad.

Con el objetivo de mejorar estos procesos, se propone el desarrollo de un sistema de administración y control de gastos comunes que permita centralizar la información, automatizar consultas y proporcionar herramientas de visualización que faciliten la toma de decisiones.

1.2. Propósito

El propósito de este Documento de Arquitectura de Software es describir la estructura arquitectónica del sistema propuesto, identificando los componentes principales, las relaciones existentes entre ellos y las decisiones de diseño adoptadas para satisfacer los requerimientos funcionales y no funcionales definidos para el proyecto.

Este documento servirá como referencia para los equipos de desarrollo, pruebas y mantenimiento, permitiendo asegurar la coherencia técnica durante todo el ciclo de vida del sistema.

Además, el documento busca justificar las decisiones arquitectónicas adoptadas utilizando como referencia la norma ISO/IEC/IEEE 42010 y el modelo arquitectónico 4+1 de Philippe Kruchten.

1.3. Ámbito

El sistema cubrirá los procesos relacionados con la administración de gastos comunes del Edificio Residencial Miraflores, incluyendo la gestión de pagos, consultas financieras, generación de reportes y visualización de información relevante para residentes,

propietarios y administradores.

El presente documento se enfoca especialmente en el módulo Consultas, Visualización y Control, responsable de proporcionar acceso a la información financiera, indicadores de gestión y herramientas de seguimiento.

Entre las funcionalidades principales contempladas se encuentran:

- Consulta de estados de cuenta.
- Visualización de historial de pagos.
- Consulta de morosidad.
- Generación de reportes financieros.
- Dashboard de indicadores.
- Seguimiento de solicitudes y reclamos.
- Exportación de información.

1.4. Definiciones, acrónimos y abreviaciones

ACRÓNIMO	DESCRIPCIÓN
<i>DAS</i>	Documento de Arquitectura de Software
<i>RF</i>	Requisito Funcional
<i>RNF</i>	Requisito No Funcional
<i>API</i>	Application Programming Interface
<i>MVC</i>	Model View Controller
<i>UI</i>	User Interface
<i>UX</i>	User Experience
<i>PDF</i>	Portable Document Format
<i>KPI</i>	Key Performance Indicator

<i>ISO</i>	International Organization for Standardization
<i>IEEE</i>	Institute of Electrical and Electronics Engineers

1.5. Resumen ejecutivo

El Sistema de Administración y Control de Gastos Comunes del Edificio Residencial Miraflores tiene como objetivo modernizar los procesos administrativos asociados a la gestión financiera de la comunidad. La solución permitirá centralizar la información, automatizar consultas y proporcionar herramientas de visualización que favorezcan la transparencia y eficiencia operativa.

La arquitectura propuesta adopta una estructura en capas complementada con el patrón MVC, permitiendo una clara separación de responsabilidades y facilitando el mantenimiento futuro del sistema.

El diseño considera principios de calidad establecidos por ISO 25010, garantizando atributos como seguridad, usabilidad, mantenibilidad y rendimiento.

1.6. Arquitectura del sistema

La arquitectura del sistema será documentada utilizando el modelo 4+1, compuesto por:

- Vista de Escenario.
- Vista Lógica.
- Vista de Desarrollo.
- Vista de Procesos.
- Vista Física.

Cada una de estas vistas permitirá representar diferentes perspectivas del sistema y facilitar la comprensión por parte de los distintos stakeholders involucrados.

2. VISIÓN DEL SISTEMA

2.1. Descripción general del sistema

El sistema corresponde a una plataforma web orientada a la administración y control de gastos comunes. Su objetivo principal es proporcionar un entorno centralizado donde los distintos usuarios puedan acceder a información relevante de manera segura y eficiente.

La plataforma permitirá a los residentes consultar su situación financiera, visualizar pagos realizados y realizar seguimiento a solicitudes. Por otra parte, los administradores podrán generar reportes, supervisar indicadores y gestionar información financiera consolidada.

2.2. Objetivos del sistema

Objetivo General

Desarrollar una solución informática que permita optimizar la gestión de gastos comunes del Edificio Residencial Miraflores mediante la automatización de procesos administrativos y financieros.

Objetivos Específicos

- Mejorar el acceso a la información.
- Incrementar la transparencia financiera.
- Reducir tiempos de atención.
- Disminuir errores asociados a procesos manuales.
- Facilitar la generación de reportes.
- Proporcionar indicadores para la toma de decisiones.
- Garantizar trazabilidad de operaciones.

2.3. Requerimientos funcionales

Número	Nombre Requisito Funcional	Descripción
--------	----------------------------	-------------

RFN°18	Consultar Estado de Cuenta	Permitir que residentes y propietarios consulten en línea el detalle actualizado de sus gastos comunes, pagos realizados y saldos pendientes.
RF N°19	Visualizar Historial de Pagos	Permitir acceder al registro histórico de pagos efectuados por una unidad habitacional.
RF N°20	Consultar Morosidad	Permitir acceder al registro histórico de pagos efectuados por una unidad habitacional.
RF N°21	Dashboard Financiero	Mostrar indicadores financieros relevantes para apoyar la gestión administrativa.
RF N°22	Generar Reportes Financieros	Permitir generar informes por períodos determinados.
RF N°23	Exportar Reportes	Permitir exportar reportes en formatos PDF y Excel.
RF N°24	Consultar Solicitudes	Permitir visualizar solicitudes registradas por residentes.
RF N°25	Seguimiento de Solicitudes	Permitir conocer el estado de avance de cada solicitud o reclamo.

2.4. Requerimientos no funcionales

Seguridad: El acceso al sistema deberá requerir autenticación mediante credenciales únicas y control de permisos según el rol del usuario.

Rendimiento: Las consultas deberán responder en un tiempo inferior a tres segundos bajo condiciones normales de operación.

Disponibilidad: El sistema deberá estar disponible al menos un 99% del tiempo durante períodos de operación.

Usabilidad: La interfaz deberá ser intuitiva y comprensible para usuarios con distintos niveles de experiencia tecnológica.

Mantenibilidad: La arquitectura deberá facilitar futuras modificaciones e incorporación de nuevas funcionalidades.

Escalabilidad: La solución deberá permitir el crecimiento progresivo de usuarios y funcionalidades.

2.5. Supuestos y dependencias

Se asume que los usuarios disponen de acceso a internet y dispositivos compatibles con navegadores modernos.

La operación del sistema dependerá de:

- Servidor de aplicaciones.
- Base de datos centralizada.
- Servicio de correo electrónico.
- Pasarela de pago (si otros módulos la incorporan).

3. ESTILOS Y PATRONES ARQUITECTÓNICOS

3.1. Estilo arquitectónico adoptado

Para el desarrollo del Sistema de Administración y Control de Gastos Comunes se adopta una Arquitectura en Capas (Layered Architecture) complementada con una organización modular basada en responsabilidades funcionales.

Esta arquitectura divide el sistema en diferentes niveles lógicos, permitiendo separar adecuadamente la presentación de la lógica de negocio y del acceso a los datos. Esta

separación facilita la mantenibilidad, la reutilización de componentes y la escalabilidad de la solución.

Las capas consideradas son:

Capa de Presentación: Corresponde a la interfaz utilizada por residentes, propietarios y administradores para interactuar con el sistema.

Entre sus responsabilidades se encuentran:

- Visualización de estados de cuenta.
- Consulta de pagos.
- Visualización de indicadores.
- Generación de reportes.
- Gestión de solicitudes.

Capa de Negocio: Contiene las reglas que gobiernan el funcionamiento del sistema.

Sus funciones principales incluyen:

- Cálculo de morosidad.
- Consolidación de información financiera.
- Validación de permisos.
- Generación de indicadores.
- Control de solicitudes.

Capa de Persistencia: Gestiona la comunicación entre la lógica de negocio y la base de datos.

Sus responsabilidades son:

- Recuperación de información.
- Almacenamiento de registros.
- Consultas complejas.
- Actualización de datos.

Capa de Datos: Corresponde a la base de datos centralizada donde se almacenan todos los registros relacionados con usuarios, pagos, gastos comunes, solicitudes y reportes.

3.2. Justificación del estilo según el contexto del sistema

La selección de una arquitectura en capas responde a las características y necesidades del proyecto.

En primer lugar, el sistema posee procesos administrativos claramente definidos, lo que permite estructurar la solución en componentes con responsabilidades específicas. Esta separación reduce el acoplamiento entre módulos y facilita la incorporación de nuevas funcionalidades sin afectar significativamente al resto de la aplicación.

Asimismo, el proyecto será desarrollado utilizando una metodología Cascada, donde la documentación y trazabilidad adquieren especial relevancia. Una arquitectura en capas favorece la comprensión del sistema y permite representar claramente las relaciones entre componentes dentro de las vistas arquitectónicas definidas por el modelo 4+1.

Desde la perspectiva de mantenimiento, esta arquitectura facilita futuras actualizaciones relacionadas con cambios normativos, modificaciones en los procesos administrativos o incorporación de nuevos servicios para la comunidad.

Finalmente, este enfoque resulta adecuado para una aplicación web orientada a la gestión de información, donde la mayor parte de las operaciones corresponden a consultas, generación de reportes y procesamiento de datos.

3.3. Patrones de diseño aplicados

Patrón MVC (Model View Controller)

El patrón MVC será utilizado para separar la interfaz de usuario de la lógica de negocio y del modelo de datos.

Modelo

Representa las entidades del sistema:

- Usuario
- Residente
- Propietario
- Pago
- Gasto Común
- Solicitud
- Reporte

Vista: Corresponde a las pantallas visualizadas por los usuarios.

Controlador: Gestiona las solicitudes realizadas desde la interfaz y coordina las operaciones necesarias para responder a dichas acciones.

Patrón Repository: Permite abstraer el acceso a datos, desacoplando la lógica de negocio de los mecanismos específicos de persistencia.

Beneficios:

- Facilita pruebas.
- Reduce dependencia de la base de datos.
- Mejora mantenibilidad.

Patrón Service Layer

Centraliza la lógica de negocio del sistema.

Ejemplos:

- Servicio de Consultas.
- Servicio de Reportes.
- Servicio de Indicadores.
- Servicio de Solicitudes.

Patrón DTO (Data Transfer Object)

Utilizado para transportar información entre capas evitando exponer directamente las entidades internas.

Esto mejora:

- Seguridad.
- Rendimiento.
- Modularidad.

4. MODELO 4 +1 Y VISTAS ARQUITECTÓNICAS

4.1. VISTA DE ESCENARIO

4.1.1. Propósito

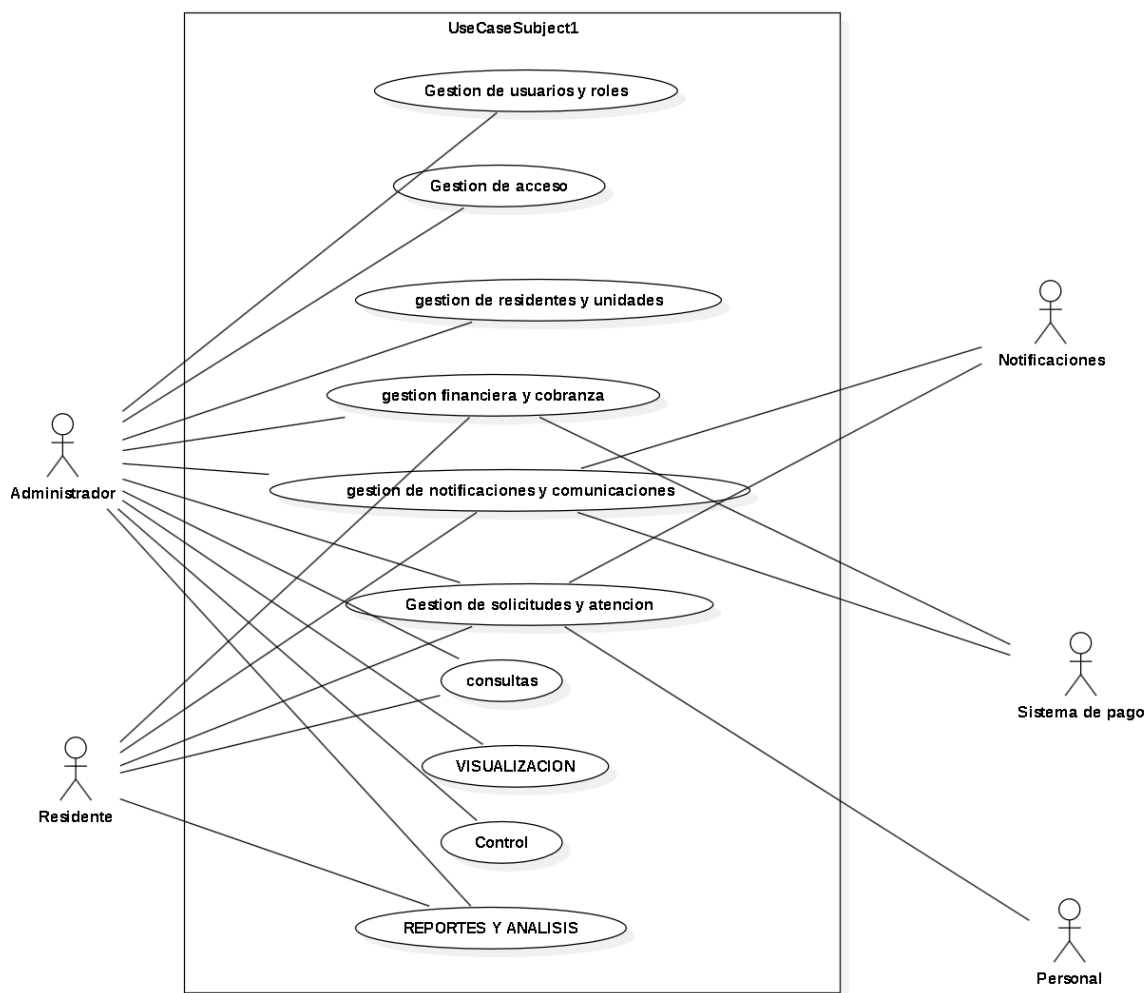
La vista de escenario tiene como finalidad representar la interacción entre los distintos usuarios y el sistema mediante casos de uso que describen situaciones reales de operación.

Esta vista permite validar que la arquitectura propuesta responde adecuadamente a las necesidades identificadas durante el levantamiento de requisitos.

4.1.2. Actores

Actores	Descripción
Residente	Usuario responsable de consultar información asociada a su unidad habitacional.
Propietario	Usuario que requiere supervisar pagos y estados financieros asociados a sus propiedades.
Administrador	Responsable de gestionar la información financiera y operacional del edificio.
Comité de Administración	Actor encargado de supervisar indicadores y reportes financieros para apoyar la toma de decisiones.

4.1.3. Diagrama general de casos de uso



4.1.4.

El diagrama general representa las principales interacciones entre los actores y el sistema.

Incluye:

- Consultar estado de cuenta.
- Consultar historial de pagos.
- Consultar morosidad.
- Visualizar dashboard.
- Generar reportes.

- Exportar información.
- Consultar solicitudes.
- Actualizar estado de solicitudes.

4.1.5. Diagrama de casos de uso específicos

(Imagen Diagrama Caso de Uso Consultar Estado de Cuenta)

(Imagen Diagrama Caso de Uso Generar Reporte Financiero)

(Imagen Diagrama Caso de Uso Dashboard Financiero)

(Imagen Diagrama Caso de Uso Consulta de Morosidad)

4.1.6. Lista de casos de uso

Código	Caso de Uso
CU-001	Consultar Estado de Cuenta
CU-002	Consultar Historial de Pagos
CU-003	Consultar Morosidad
CU-004	Visualizar Dashboard Financiero
CU-005	Generar Reporte Financiero
CU-006	Exportar Reporte
CU-007	Consultar Solicitudes
CU-008	Actualizar Estado Solicitud

4.1.7. Especificación de casos de uso

Caso de Uso	Consultar Estado de Cuenta	Identificador: CU-001
Actores	Residente, Propietario	
Tipo	Primario	
Referencias	Requisito Funcional N°18	
Precondición	El usuario debe encontrarse autenticado en el sistema.	
Postcondición	El sistema presenta la información financiera actualizada correspondiente a la unidad habitacional seleccionada.	
Descripción	Permite visualizar gastos comunes, pagos realizados, deudas pendientes y cargos extraordinarios.	
Resumen	[Resumen de alto nivel del funcionamiento]	

CURSO NORMAL

Paso	Ejecutor	Actividad
1	Usuario	Accede al módulo de consultas
2	Usuario	Selecciona Estado de Cuenta
3	Sistema	Recupera información financiera
4	Sistema	Procesa datos
5	Sistema	Muestra resultados

CURSO ALTERNATIVO

Paso	Descripción
3A	No existen registros asociados
3B	Se produce error de conexión

5A	Usuario no posee permisos
----	---------------------------

4.2. VISTA LÓGICA

4.2.1. Propósito

La Vista Lógica describe la organización funcional del sistema y las relaciones existentes entre las principales entidades que participan en los procesos de negocio.

Su objetivo es representar la estructura conceptual de la solución desde la perspectiva de los desarrolladores y analistas.

4.2.2. Diagrama de clases

(Imagen Diagrama de Clases)

4.2.3. Descripción diagrama de clases

Las principales entidades identificadas son:

Usuario: Representa a cualquier persona que accede al sistema.

Residente: Especialización de Usuario asociada a una unidad habitacional.

Propietario: Representa al dueño de una o más unidades.

Unidad Habitacional: Contiene información relacionada con departamentos o estacionamientos.

Estado Cuenta: Almacena la situación financiera de cada unidad.

Pago: Representa pagos registrados.

Gasto Común: Representa cobros periódicos.

Reporte Financiero: Consolida información estadística y financiera.

Solicitud: Gestiona requerimientos y reclamos.

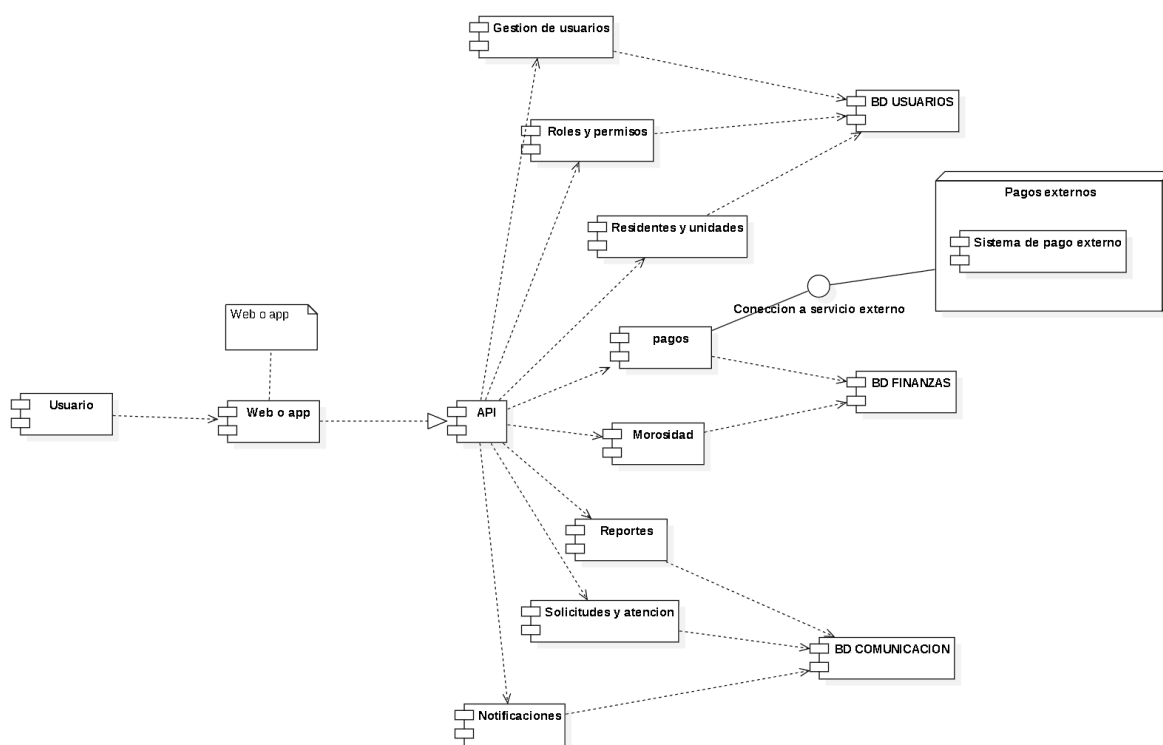
Las relaciones entre estas entidades permiten representar adecuadamente la operación financiera y administrativa del edificio.

4.3. VISTA DE IMPLEMENTACIÓN/DESARROLLO

4.3.1. Propósito

Representar la organización física del software desde la perspectiva del desarrollo, identificando componentes, módulos y dependencias.

4.3.2. Diagrama de componente



4.3.3. Descripción diagrama de componente

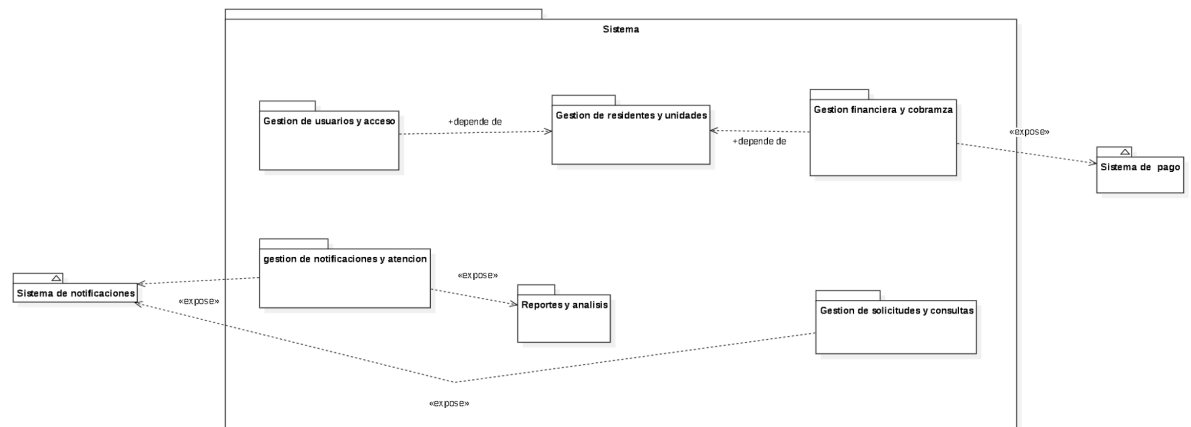
Los principales componentes son:

- Gestión de Usuarios.
- Gestión de Pagos.
- Gestión de Gastos Comunes.
- Gestión de Solicitudes.

- Consultas y Reportes.
- Dashboard e Indicadores.
- Notificaciones.

Cada componente mantiene responsabilidades claramente definidas y se comunica mediante interfaces bien establecidas.

4.3.4. Diagrama de paquete



4.3.5. Descripción diagrama de paquete

La organización modular contempla paquetes orientados a dominio:

- usuarios
- pagos
- gastos comunes
- solicitudes
- consultas
- reportes
- dashboard

- seguridad

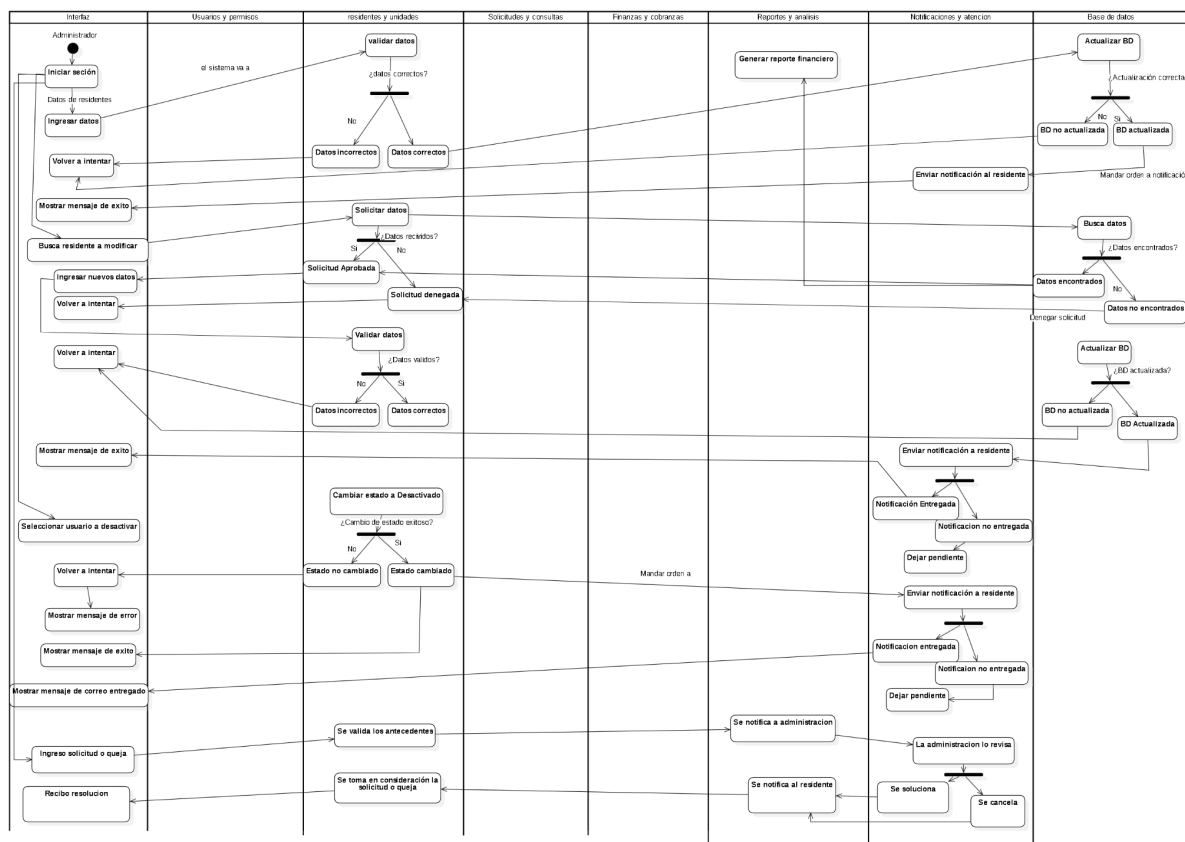
Esta estructura favorece la mantenibilidad y escalabilidad del sistema.

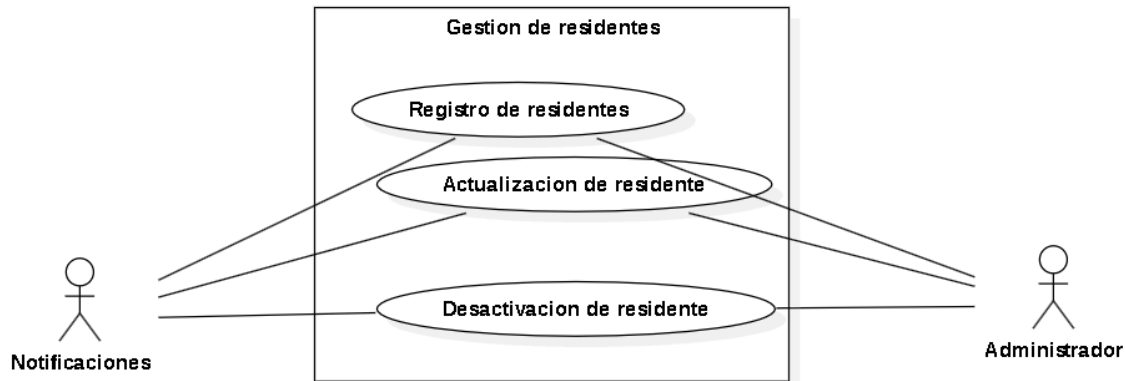
4.4. VISTA DE PROCESOS

4.4.1. Propósito

Representar los procesos dinámicos que ocurren durante la ejecución del sistema, mostrando el flujo de actividades y la interacción entre usuarios y componentes.

4.4.2. Diagrama de actividad





4.4.3. Descripción diagrama de actividad

El proceso comienza cuando un usuario autenticado accede al módulo de consultas.

Posteriormente selecciona la información que desea visualizar. El sistema valida permisos, recupera datos desde la base de datos y procesa la información necesaria para generar la respuesta correspondiente.

Finalmente, los resultados son presentados mediante tablas, indicadores o reportes exportables.

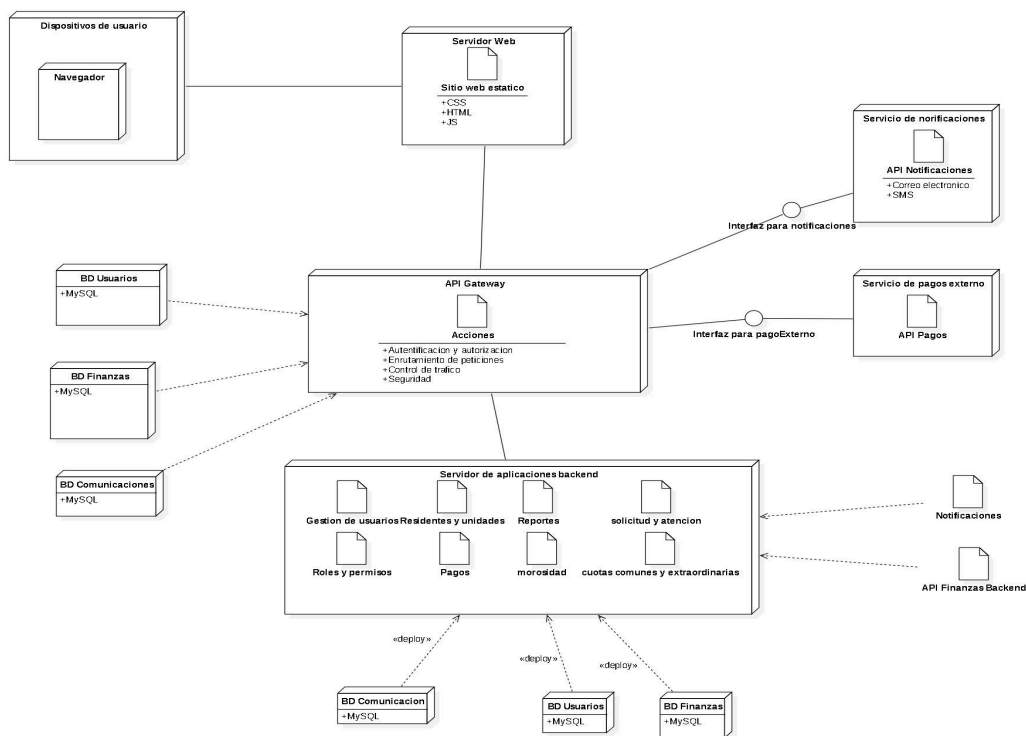
Este flujo garantiza la consistencia de la información y asegura que únicamente usuarios autorizados puedan acceder a datos sensibles.

4.5. VISTA FÍSICA

4.5.1. Propósito

Representar la infraestructura tecnológica necesaria para soportar la ejecución del sistema.

4.5.2. Diagrama de despliegue



4.5.3. Descripción diagrama de despliegue

La arquitectura física contempla tres niveles principales:

Cliente: Navegadores web utilizados por residentes, propietarios y administradores.

Servidor de Aplicaciones: Responsable de ejecutar la lógica de negocio y coordinar las operaciones del sistema.

Base de Datos: Almacena toda la información relacionada con usuarios, pagos, gastos comunes, solicitudes y reportes.

Adicionalmente se contempla la integración con servicios externos como correo electrónico y plataformas de pago cuando corresponda a otros módulos del proyecto.

5. REQUISITOS DE CALIDAD

5.1. Propósito

El propósito de esta sección es establecer los atributos de calidad que deberá cumplir la solución propuesta para garantizar que el sistema no solo satisfaga los requerimientos funcionales definidos, sino que además entregue una experiencia de uso adecuada, segura y sostenible en el tiempo.

La definición de estos atributos se fundamenta en el modelo de calidad ISO/IEC 25010, reconocido internacionalmente como referencia para la evaluación de productos de software. Considerando que el sistema administrará información financiera sensible relacionada con pagos, morosidad y gastos comunes, resulta fundamental asegurar que la solución proporcione altos niveles de seguridad, disponibilidad y confiabilidad.

Asimismo, debido a que el Módulo de Consultas, Visualización y Control constituye uno de los principales puntos de interacción entre los usuarios y el sistema, los atributos relacionados con usabilidad, rendimiento y accesibilidad adquieren especial relevancia para el éxito del proyecto.

5.2. Atributos de calidad

Atributo de Calidad	Descripción	Justificación

Usabilidad	El sistema deberá ser intuitivo, fácil de aprender y utilizar para cualquier usuario.	Muchos residentes poseen distintos niveles de conocimiento tecnológico, por lo que una interfaz clara facilitará la adopción de la plataforma.
Accesibilidad	Las interfaces deberán considerar principios de accesibilidad digital compatibles con WCAG 2.1 nivel AA.	Garantiza igualdad de acceso a la información para todos los usuarios.
Rendimiento	El sistema deberá responder oportunamente ante consultas y generación de reportes.	El módulo de consultas requiere acceso rápido a información financiera actualizada.
Seguridad	La información financiera deberá estar protegida mediante mecanismos de autenticación y autorización.	El sistema administrará datos sensibles relacionados con pagos y morosidad.
Fiabilidad	El sistema deberá mantener un comportamiento consistente y estable durante su operación.	Evita pérdida de información y asegura continuidad del servicio.
Disponibilidad	La plataforma deberá encontrarse operativa la mayor parte del tiempo.	Los usuarios podrán consultar información en cualquier momento.
Mantenibilidad	La arquitectura deberá facilitar futuras modificaciones y correcciones.	Se espera que el sistema evolucione incorporando nuevas funcionalidades.
Escalabilidad	El sistema deberá soportar crecimiento de usuarios y módulos.	Facilita la incorporación de nuevos procesos administrativos.

Portabilidad	La solución deberá ejecutarse en distintos navegadores y entornos tecnológicos.	Permite mayor compatibilidad con dispositivos de los usuarios.
--------------	---	--

5.3. Reglas y criterios de evaluación de calidad

Para verificar el cumplimiento de los atributos definidos se establecerán criterios medibles y mecanismos de validación.

Usabilidad

Criterio:

- Los usuarios deberán completar las principales tareas sin asistencia externa.

Método de evaluación:

- Pruebas de usuario.
- Evaluación heurística de Nielsen.

Rendimiento

Criterio:

- Tiempo de respuesta inferior a 3 segundos para consultas comunes.
- Generación de reportes inferior a 10 segundos.

Método de evaluación:

- Pruebas de carga.
- Pruebas de rendimiento.

Seguridad

Criterio:

- Acceso restringido según rol.
- Protección de credenciales.
- Registro de actividades relevantes.

Método de evaluación:

- Revisión de controles de acceso.
- Pruebas funcionales de seguridad.

Accesibilidad

Criterio:

- Cumplimiento de recomendaciones WCAG 2.1 nivel AA.

Método de evaluación:

- Inspección de interfaces.
- Validadores automáticos de accesibilidad.

Mantenibilidad

Criterio:

- Modularidad y separación adecuada de responsabilidades.

Método de evaluación:

- Revisión arquitectónica.
- Inspección de componentes.

6. PRINCIPIOS DE DISEÑO APLICADOS

6.1. Propósito

Esta sección describe los principios de diseño utilizados durante la construcción de la arquitectura del sistema. Su aplicación busca asegurar una solución organizada, escalable y fácil de mantener, alineada con las buenas prácticas de ingeniería de software.

Los principios seleccionados permiten reducir la complejidad del sistema y facilitan futuras modificaciones, aspecto especialmente importante considerando que el proyecto será desarrollado de forma modular por distintos equipos.

6.2. Principios de diseño

Principio	Descripción	Aplicación en el Sistema
Cohesión	Cada componente debe cumplir una responsabilidad específica.	El módulo de consultas se enfoca exclusivamente en visualización, reportes e indicadores.
Bajo Acoplamiento	Minimiza dependencias entre módulos.	Consultas obtiene información mediante servicios sin depender directamente de la lógica de otros módulos.
Encapsulamiento	Oculto detalles internos de implementación.	Las operaciones de acceso a datos se abstraen mediante repositorios y servicios.
Modularidad	División del sistema en componentes independientes.	Cada grupo de trabajo desarrolla funcionalidades específicas sin afectar otros módulos.

Abstracción	Simplifica la representación de elementos complejos.	Los usuarios interactúan con reportes e indicadores sin conocer la lógica interna de procesamiento.
Reutilización	Favorece el uso compartido de componentes.	Los servicios de consultas y reportes pueden ser utilizados por otros módulos futuros.
Separación de Responsabilidades	Cada capa realiza funciones claramente definidas.	Presentación, negocio y persistencia mantienen responsabilidades independientes.

7. PROTOTIPO

7.1. Propósito

El prototipo tiene como finalidad representar visualmente el comportamiento esperado del sistema antes de su implementación definitiva. Permite validar requisitos, mejorar la comunicación entre los participantes del proyecto y detectar oportunidades de mejora tempranamente.

Para el presente proyecto se utilizará un prototipo de alta fidelidad desarrollado en Figma, permitiendo simular la navegación entre pantallas y validar aspectos relacionados con la experiencia de usuario.

Particularmente para el Módulo de Consultas, Visualización y Control, el prototipo permitirá validar la disposición de información financiera, reportes e indicadores antes de la etapa de implementación.

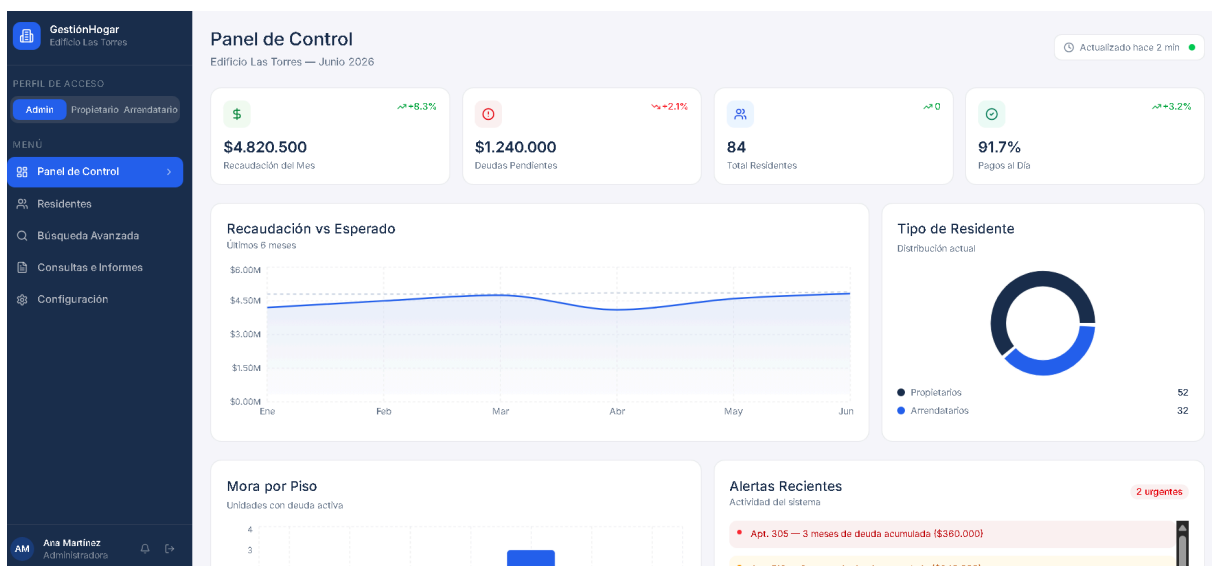
7.2. Mockups (imágenes con una breve descripción)

Pantalla de Inicio de Sesión

Descripción:

Permite a residentes, propietarios y administradores autenticarse en la plataforma utilizando sus credenciales de acceso.

Dashboard Principal



Descripción:

Presenta indicadores generales relacionados con pagos, morosidad y estado financiero de la comunidad.

Consulta de Estado de Cuenta

Constructor de Criterios

Campo: Estado de cuenta

Condición: es igual a

Valor: Al día

+ Añadir criterio

Generar Informe

Consultas Guardadas

- Residentes en mora (≥1 mes)
1 criterio
- Deudas críticas
1 criterio
- Arrendatarios al día
0 criterios

INFORME GENERADO

Resultados de Consulta

Edificio Las Torres - 14 Jun 2026, 09:45 p. m.

Estado de cuenta es igual a "Al día"

9
Residentes encontrados

\$0
Deuda total acumulada

0.0 m
Promedio mora por residente

9
Sin deuda de 9

9 registros

RESIDENTE	APTO.	TIPO	ESTADO	MESES MORA	DEUDA	ÚLTIMO PAGO
Ana García Fuentes	101	Propietario	Al día	—	\$0	27 may 2026
Tomás Ríos Lagos	401	Arrendatario	Al día	—	\$0	29 may 2026
Valentina Bravo Núñez	502	Propietario	Al día	—	\$0	31 may 2026
Fernanda López Cruz	601	Propietario	Al día	—	\$0	31 may 2026
Camila Torres Núñez	803	Propietario	Al día	—	\$0	28 may 2026
Diego Morales Espinoza	901	Arrendatario	Al día	—	\$0	30 may 2026

Descripción:

Permite visualizar el detalle de gastos comunes, pagos realizados y saldos pendientes.

Historial de Pagos:

Mi Estado de Cuenta

Información en tiempo real · RF-17, RF-18

Descargar Informe

RESIDENTE

Roberto Cruz Valdivia

Apto. 504 — Piso 5 - Arrendatario

RUT: 12.345.678-9

SALDO ADEUDADO

-\$290.000

Requiere pago

Próximo vencimiento

30 jun 2026

\$120.000

Cuotas vencidas

3 cuotas

\$290.000 total

Tiene deuda pendiente por \$290.000

Tiene cuotas vencidas. El no pago puede generar intereses y restricciones en áreas comunes.

Pagar ahora

Historial de Pagos y Cargos

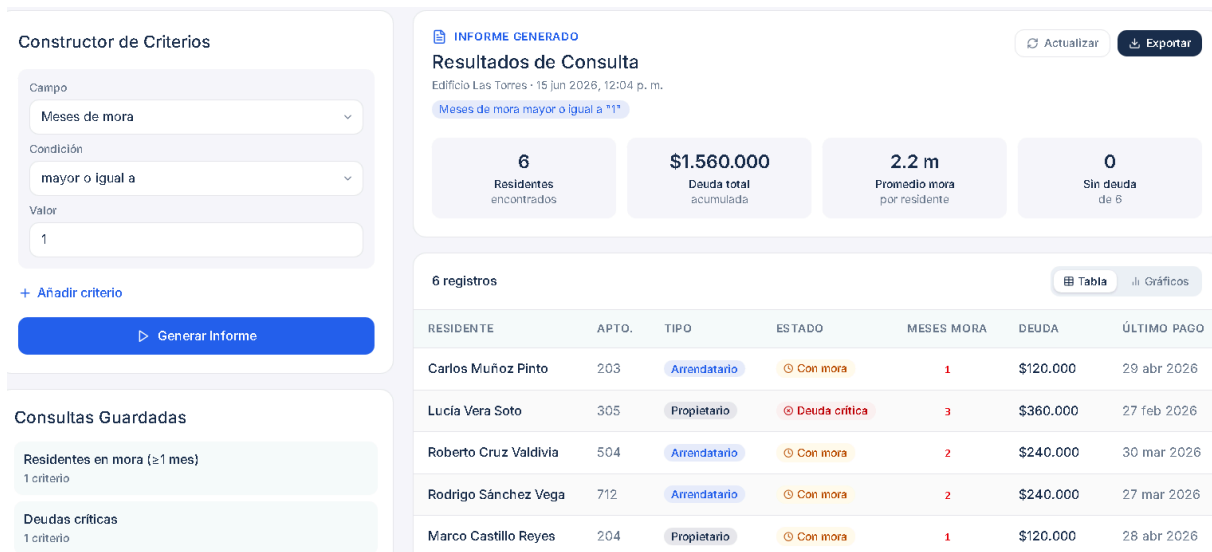
7 registros

Nº DOC.	CONCEPTO	MONTO	VENCIMIENTO	FECHA PAGO	ESTADO
GC-9624	Gastos Comunes — Junio 2026	\$120.000	31 may 2026	27 may 2026	Pagado
GC-9524	Gastos Comunes — Mayo 2026	\$120.000	30 abr 2026	29 abr 2026	Pagado
GC-9424	Gastos Comunes — Abril 2026	\$120.000	31 mar 2026	—	Vencido
GC-9324	Gastos Comunes — Marzo 2026	\$120.000	28 feb 2026	—	Vencido

Descripción:

Muestra el registro histórico de pagos efectuados por una unidad habitacional.

Reportes Financieros



Descripción:

Permite generar y exportar reportes administrativos para análisis financiero.

Gestión de Solicitudes

(Imagen Prototipo Solicitudes)

Descripción:

Facilita el seguimiento de solicitudes y reclamos registrados por los residentes.

7.3. Justificar herramientas de prototipado

Se seleccionó Figma como herramienta de prototipado debido a que proporciona múltiples

ventajas para el desarrollo colaborativo del proyecto.

Entre sus principales beneficios destacan:

- Trabajo colaborativo en tiempo real.
- Facilidad para compartir diseños.
- Creación de prototipos interactivos.
- Integración con procesos de diseño UX/UI.
- Amplia adopción en la industria.

Su utilización permite validar tempranamente la experiencia de usuario y disminuir riesgos asociados a errores de diseño durante etapas posteriores.

8. EVALUACIÓN DE CALIDAD HEURÍSTICA DE NIELSEN

8.1. Propósito

La evaluación heurística tiene como objetivo identificar problemas potenciales de usabilidad en las interfaces del sistema antes de su implementación definitiva.

Para ello se utilizarán las diez heurísticas propuestas por Jakob Nielsen, ampliamente reconocidas como referencia para la evaluación de interfaces de usuario.

Esta evaluación resulta especialmente relevante para el Módulo de Consultas y Visualización, ya que concentra gran parte de la interacción entre usuarios y sistema.

8.2. Lista de verificación

1. Visibilidad del estado del sistema: El sistema informa claramente cuando una consulta está siendo procesada o un reporte se encuentra en generación.
2. Correspondencia entre sistema y mundo real a lo cual utilizan conceptos familiares como:
 - Estado de Cuenta.

- Historial de Pagos.
- Gastos Comunes.
- Morosidad.

3. Control y libertad del usuario: El usuario puede regresar a pantallas anteriores o cancelar operaciones.

4. Consistencia y estándares: Todas las interfaces mantienen diseño uniforme.

5. Prevención de errores: Se incorporan validaciones antes de ejecutar acciones relevantes.

6. Reconocimiento antes que memoria: La información importante permanece visible durante la navegación.

7. Flexibilidad y eficiencia: Los usuarios pueden acceder rápidamente a consultas frecuentes.

8. Diseño minimalista: Se evita presentar información irrelevante.

9. Ayuda para reconocer errores: Los mensajes indican claramente causas y posibles soluciones.

10. Ayuda y documentación: Se incorporan mensajes de apoyo y orientación.

8.3. Análisis y métricas de resultados

Los resultados obtenidos serán clasificados según nivel de severidad:

Severidad	Descripción
0	No se detecta problema
1	Problema cosmético
2	Problema menor

3	Problema importante
4	Problema crítico

Se espera obtener una puntuación promedio inferior a 2, indicando un nivel adecuado de usabilidad para la primera versión del sistema.

9. CONTROL DE VERSIONES

9.1. Propósito

Establecer un mecanismo formal para gestionar cambios realizados durante el desarrollo del proyecto, permitiendo mantener trazabilidad sobre las modificaciones efectuadas y facilitando la colaboración entre los distintos equipos responsables de cada módulo.

9.2. Control de versión utilizado

Se utilizará un esquema de versionamiento semántico.

Formato:

MAJOR.MINOR.PATCH

Ejemplos:

- 1.0.0
- 1.1.0
- 1.1.1

Versión Mayor (MAJOR)

Cambios significativos que modifican funcionalidades principales.

Versión Menor (MINOR)

Incorporación de nuevas funcionalidades compatibles con versiones anteriores.

Corrección (PATCH)

Corrección de errores sin afectar funcionalidades existentes.

La utilización de versionamiento semántico facilita la identificación del impacto de cada actualización y es ampliamente utilizada en proyectos profesionales.

Versión 0.0.1:

Esta versión era la ventana principal sin refinar las funcionalidades de cada perfil para tener una plantilla sobre la cual trabajar

0.1.3:

Esta versión añade una ventana de login para el prototipo que cambia según administrador, residente, arrendatario o propietario como un prototipo funcional

0.2.3

Esta versión fue la final que se usó como versión de prototipo para entrega que arregla errores de código, busca y cumple los requisitos estipulados para el apartado de generar informes y tiene ventana de login que separa cada ámbito del sistema.

9.3. Justificar herramientas de versionamiento

Se propone utilizar Git como sistema de control de versiones junto con GitHub como plataforma de almacenamiento y colaboración.

Beneficios:

- Historial completo de cambios.
- Trabajo colaborativo.
- Gestión de ramas.
- Recuperación de versiones anteriores.

- Control de conflictos.
- Integración con herramientas de desarrollo modernas.

Esta combinación resulta adecuada para proyectos modulares desarrollados por múltiples equipos.

10. CONCLUSIONES

La arquitectura propuesta para el Sistema de Administración y Control de Gastos Comunes del Edificio Residencial Miraflores proporciona una solución alineada con las necesidades identificadas durante el análisis del problema.

La adopción de una arquitectura en capas complementada con el patrón MVC permite mantener una adecuada separación de responsabilidades, favoreciendo la mantenibilidad y evolución futura del sistema.

La utilización del modelo arquitectónico 4+1 facilita la representación de la solución desde distintas perspectivas, permitiendo comprender tanto la estructura interna como las interacciones entre usuarios y componentes.

Particularmente, el Módulo de Consultas, Visualización y Control constituye un elemento clave para mejorar la transparencia administrativa de la comunidad, permitiendo a residentes, propietarios y administradores acceder oportunamente a información financiera relevante mediante estados de cuenta, reportes e indicadores de gestión.

Finalmente, la incorporación de estándares como ISO/IEC/IEEE 42010, ISO 25010 y las heurísticas de Nielsen permite asegurar una solución orientada a la calidad, la usabilidad y la sostenibilidad en el tiempo.

11. BIBLIOGRAFÍA

- ISO/IEC/IEEE 42010: Systems and Software Engineering – Architecture Description.
- ISO/IEC 25010: Systems and Software Quality Requirements and Evaluation.
- Kruchten, P. (1995). The 4+1 View Model of Architecture.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. Design Patterns: Elements of Reusable Object-Oriented Software.
- Nielsen, J. (1994). Usability Engineering.
- Pressman, R. Ingeniería de Software: Un enfoque práctico.
- Sommerville, I. Ingeniería de Software.
- Documentación oficial de Figma.
- Documentación oficial de Git y GitHub.

12. ANEXOS

12.1. Carta Gantt: [Carta Gantt](#)